

Welcome to Fine-Tuning

What is Fine-Tuning?

- Customizing a pre-trained model for specific tasks
- Creating your own private version of a model
- Teaching the model your specific style, format, or domain



KEY POINT

Fine-tuning adapts existing knowledge, not teaching from scratch

LoRA - Low Rank Adapters

What is LoRA?

- **Lo**w **R**ank **A**dapters
- Trains a small set of parameters, not the entire model
- These parameters "nudge" the big model's behavior



ANALOGY

Like adding a specialized lens to a camera — modifies output without rebuilding the camera

ⓘ Note: OpenAI likely uses LoRA internally (not disclosed)

Three Stages of Fine-Tuning

1 CREATE DATA

Prepare Training Data

- Format data as JSONL (JSON Lines)
- Each line represents exactly one training example
- Upload prepared file to OpenAI

2 RUN TRAINING

Execute Job

- Monitor loss metrics closely
- Watch accuracy improve over epochs
- Training happens securely in OpenAI's cloud

3 EVALUATE

Iterate & Improve

- Test the fine-tuned model against validation set
- Tweak hyperparameters based on performance
- Repeat the process if needed

OpenAI Fine-Tuning Options

01 **SFT** - Supervised Fine-Tuning
(most common)

02 **DPO** - Direct Preference Optimization

03 **RFT** - Reinforcement Fine-Tuning



OUR FOCUS

SFT (Supervised Fine-Tuning)



Supervised Fine-Tuning (SFT)

⚙️ What It Does

- Provide input → correct output pairs
- Model learns from labeled examples
- Customizes style, tone, and format

★ Best For

- Classification tasks
- Translation with nuance
- Generating content in a specific style
- Correcting consistent failures

SUPPORTED MODELS

GPT-4.1

GPT-4.1-mini

GPT-4.1-nano

Direct Preference Optimization (DPO)

👍 What It Does

- Learn from good vs. bad examples
- Users give thumbs up/down feedback
- Refines based on preferences

⚙️ Best For

- Adjusting style and tone
- Subjective improvements
- Nuanced behavior changes

CONTEXT

Successor to: RLHF (Reinforcement Learning from Human Feedback)

Reinforcement Fine-Tuning (RFT)

What It Does

- Uses a "grader" to evaluate responses
- Grader can be another LLM ("LLM as judge")
- Shifts behavior based on grades

Best For

- Expert-level performance
- Nuanced objectives
- When you don't have labeled data

 **REQUIREMENT** *Only works with reasoning models (e.g., o1-mini)*

When to Use Each Type

TYPE	USE WHEN	EXAMPLE
SFT	You have labeled data	Product classification
DPO	You have preference data	Style refinement
RFT	You can grade outputs	Medical diagnosis



KEY INSIGHT

Most Common: **SFT (Supervised Fine-Tuning)**

What You Get

Your Private Model

- Custom version of base model
- Only accessible to you
- Small set of parameters stored
- Applied to base model at inference

✗ **NOT:** A complete copy of GPT

✓ **ACTUALLY:** Small adapter parameters (LoRA)

Data Format - JSONL

JSON LINES FORMAT

 training.jsonl

```
{"messages": [{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]}\n{"messages": [{"role": "user", "content": "..."}, {"role": "assistant", "content": "..."}]}
```

Anatomy of Each Line

-  One complete training example
-  User message (INPUT)
-  Assistant message (EXPECTED OUTPUT)



API COMPATIBILITY

Similar to: Batch API format

How Much Data?

📌 OpenAI Recommendation

- Start with **50-100 examples**
- Only increase if strictly needed
- More $\neq$ always better

❓ Why So Few?

- Frontier models already possess massive knowledge
- Fine-tuning adds "hints" rather than core knowledge
- Teaching specific formats or styles requires very few examples



Training Process

1

Upload training file

Prepare file for fine-tuning purpose

```
client.files.create(  
    file=open("training.jsonl", "rb"),  
    purpose="fine-tune"  
)
```

2

Create fine-tuning job

Launch job with specific model

```
client.fine_tuning.jobs.create(  
    training_file=file_id,  
    model="gpt-4.1-nano"  
)
```

3

Monitor and retrieve

Wait for completion & validation

○ Running...



✓ Succeeded

Key Hyperparameters

Epochs

- Number of passes through the training data
- Usually 1 epoch is enough (with lots of data)

Learning Rate

- Controls how much to adjust parameters
- OpenAI auto-selects (usually)

Batch Size

- Number of examples per training step
- Smaller = more precise, slower
- Larger = faster, less precise

Seed

- Ensures reproducibility of results
- Critical for comparing experiments

Monitoring Training

- **Loss Metrics**

Training Loss: How well model fits training data

Validation Loss: How well model generalizes

- **What to Look For**

- Loss decreasing over time
- Validation loss not increasing (overfitting)

- **OpenAI Dashboard**

Provides visual charts of progress during the training job



PLATFORM

<https://platform.openai.com/finetune>

Validation Data

🕒 Purpose

- Held-out data not used for training
- Unbiased measure of performance
- Detects overfitting

📋 Recommendation

- 50-100 validation examples
- Separate from training data
- Same format as training data

When Fine-Tuning Works Well

CHECKLIST

Good Use Cases



Setting style or tone

Aligning model voice with brand identity



Formatting

E.g., ensuring strict JSON or XML output



Handling edge cases

Managing rare or complex scenarios



Improving output reliability

Reducing hallucinations in known domains



Correcting specific failures

Fixing persistent logic or knowledge gaps



Hard-to-explain tasks

When "show, don't tell" works better than prompts

When Fine-Tuning Doesn't Work

AVOID Poor Use Cases

- ✗ Teaching completely new knowledge
- ✗ Tasks requiring massive domain expertise
- ✗ When prompt engineering works well
- ✗ When you have limited examples



BETTER ALTERNATIVE

Use RAG or larger context

Cost Considerations

Pricing (approximate)

- **Training:** ~\$0.008 per 1K tokens (nano)
- **Inference:** Slightly more than base model
- **Storage:** Free for limited time

Cost Examples

- 100 examples $\approx$ \$0.10
- 20,000 examples $\approx$ \$3-4



TIP

Start small, scale if needed

The Fine-Tuning API Flow

1 PREPARE DATA

JSONL format

2 UPLOAD FILES

Training & validation

3 CREATE JOB

Specify model &
hyperparameters

4 MONITOR

Watch loss metrics

5 RETRIEVE MODEL

Get fine-tuned model ID

6 TEST

Run inference

7 ITERATE

Adjust and repeat

Testing Your Model

STEP 1 Use Same Format

```
test_model.py

response = client.chat.completions.create(
    model="ft:gpt-4.1-nano:personal:suffix:id",
    messages=[
        {"role": "user", "content": "..."}
    ]
)
```

i Ensure the model ID matches exactly what was returned from the training job.



Important

Don't include the assistant response in your test messages!



Common Mistake

Including the answer in the test input.

Result: Perfect scores (False Positive) because the model just reads the answer provided.

Evaluation Metrics



For Regression

Predicting numbers

- **MAE**
Mean Absolute Error
- **RMSE**
Root Mean Squared Error



For Classification

Categorizing inputs

- Accuracy
- Precision / Recall
- F1 Score



For Generation

Creating text/content

- Human evaluation
- LLM-as-judge
- Task-specific metrics

LLM FOCUS

Interpreting Loss Charts

- **Initial Drop:** Model learns format quickly
- **Plateau:** Learning slows down
- **Continued Decrease:** Model improving
- **Increase:** Possible overfitting
- **Noisy/Bumpy:** Normal variation



TIP

Use smoothing and log scale to see trends clearly

Common Issues



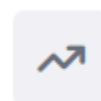
PROBLEM

Loss not decreasing



SOLUTION

More data, adjust learning rate



PROBLEM

Validation loss increasing



SOLUTION

Reduce epochs, add regularization



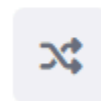
PROBLEM

Poor test performance



SOLUTION

Check data quality, try more examples



PROBLEM

Inconsistent outputs



SOLUTION

More diverse training examples

Best Practices

- 01 **Start Small** - 50-100 examples first
- 02 **Quality > Quantity** - Clean, diverse data
- 03 **Validate** - Always use validation set
- 04 **Monitor** - Watch training metrics
- 05 **Test Thoroughly** - Use held-out test set
- 06 **Iterate** - Experiment with hyperparameters
- 07 **Document** - Track what works

Data Quality Matters

DO THIS

Good Training Data



- ✓ Diverse examples
- ✓ Consistent format
- ✓ Accurate labels
- ✓ Representative of use case

AVOID THIS

Bad Training Data



- ✗ Repetitive examples
- ✗ Inconsistent format
- ✗ Incorrect labels
- ✗ Biased samples

Frontier vs. Open Source

OPTION A

Frontier Models



- ⚡ API-based (easy)
- 🔒 Limited control
- 🏰 Best for style/format
- 💰 Expensive at scale

OPTION B

Open Source



- ⚙️ Full control
- 💡 Can teach new knowledge
- 👤⚙️ Requires more expertise
- 📈 Cost-effective at scale

Real-World Example

CASE STUDY #01

EXPERIMENT SETUP

 TASK

Product price estimation

 APPROACH

SFT with product descriptions → prices

 DATA VOLUME

20,000 examples



Mixed Results

OUTCOME

Fine-tuning proved not ideal for this specific task. The model struggled to generalize pricing logic from descriptions alone.

— KEY LESSON

Fine-tuning frontier models works best for **style & format**, not core knowledge.

Why It Didn't Work

⚠ The Problem

- Trying to teach specific domain knowledge
- Frontier model already has general knowledge
- Task better suited for open source fine-tuning



WHAT WORKS BETTER

- Prompt engineering with examples
- RAG with product database
- Fine-tuning smaller open source model

Experiments Are Learning

Data Science Reality

- Not all experiments succeed
- Failures eliminate approaches
- Iteration is the process
- R&D requires exploration



MINDSET

Failed experiments = valuable learning

 *Unlike Software: Bugs are bad, but failed experiments are progress*

Alternative: Deep Neural Networks

When Fine-Tuning Fails

- Build custom neural network
- Train from scratch on your data
- Full control over architecture

Trade-offs

- Requires more expertise
- Longer training time
- Can achieve excellent results



EXAMPLE

289M parameter network → competitive with frontier models

Key Takeaways

01 **Fine-tuning**
adapts pre-trained models with small parameter changes

02 **SFT**
is most common (labeled data)

03 **Start small**
(50-100 examples)

04 **Best for**
style, format, edge cases

05 **Not ideal for**
teaching new core knowledge

06 **Monitor**
loss metrics during training

07 **Iterate**
based on results

Resources



OpenAI Documentation



Fine-tuning Guide

<https://platform.openai.com/docs/guides/fine-tuning>



Dashboard

<https://platform.openai.com/finetune>



API Reference



API Docs

<https://platform.openai.com/docs/api-reference/fine-tuning>



Pricing



Pricing Page

<https://openai.com/api/pricing/>



Community



OpenAI Forum

<https://community.openai.com/>

Practical Tips

01 **Validate your JSONL** before uploading

02 **Use suffix parameter** for easy identification

03 **Set seed=42** for reproducibility

04 **Monitor in dashboard** for visual feedback

05 **Save model IDs** for later use

06 **Test incrementally** as you iterate

07 **Compare to baseline** (unfine-tuned model)

Next Steps



Practice

Hands-on experimentation

- Fine-tune with 20-100 examples
- Experiment with hyperparameters
- Try different use cases



Explore

Advanced techniques

- DPO for preference learning
- RFT for reasoning tasks
- Open source fine-tuning (more control)



Apply

Real-world implementation

- ✓ Identify your use case
- 🗄️ Prepare quality data
- 🔄 Iterate and improve

ACTION

Challenge

Your Task:

01 **Choose a use case**
(style / format / edge case)

02 **Prepare 50-100 training examples**

03 **Create validation set**

04 **Fine-tune GPT-4.1-nano**

05 **Evaluate results**

06 **Iterate with different hyperparameters**



GOAL

Understand when fine-tuning helps vs. hurts



Questions to Consider

- ❓ When should you fine-tune vs. use RAG?
- ❓ How do you know if you have enough data?
- ❓ What metrics matter for your use case?
- ❓ How do you balance cost vs. performance?
- ❓ When is open source fine-tuning better?

PRESENTATION COMPLETE

Thank You!

We've explored the landscape of fine-tuning frontier models. Now you're ready to build.

COMING NEXT



Open Source Fine-Tuning

For full control & ownership

✓ What We Covered

- ☰ Three types of fine-tuning: **SFT, DPO, RFT**
- ▼ Identifying **when to fine-tune** vs. prompt engineering
- 🔗 The complete **end-to-end workflow** & API usage
- 📈 Critical **monitoring metrics** and evaluation strategies
- 📖 **Best practices** to avoid common pitfalls



FINAL THOUGHT

"Fine-tuning is a powerful tool, not always the solution. Start simple, measure often, and iterate."

Repo Link

smqd19/
TechGadgets_Support_B...



 1
Contributor

 0
Issues

 0
Stars

 0
Forks



smqd19/TechGadgets_Support_Bot_Fine_Tuning

Contribute to smqd19/TechGadgets_Support_Bot_Fine_Tuning development by creating an account on GitHub.

 GitHub

https://github.com/smqd19/TechGadgets_Support_Bot_Fine_Tuning.git